

Design Pattern and Principal

Exercise 1: Implementing the Singleton Pattern

```
public class SingletonTest {  
    Run | Debug  
    public static void main(String[] args) {  
        // Get Logger instances  
        Logger logger1 = Logger.getInstance();  
        Logger logger2 = Logger.getInstance();  
  
        // Use logger  
        logger1.log(message: "Application started.");  
        logger2.log(message: "Processing data.");  
  
        // Verify both references point to the same object  
        System.out.println("logger1 hashCode: " + logger1.hashCode());  
        System.out.println("logger2 hashCode: " + logger2.hashCode());  
  
        if (logger1 == logger2) {  
            System.out.println(x: "Only one Logger instance exists.");  
        } else {  
            System.out.println(x: "Multiple Logger instances exist.");  
        }  
    }  
}
```

```
public class Logger {  
    // Private static instance of Logger  
    private static Logger instance;  
  
    // Private constructor to prevent object creation from outside  
    private Logger() {  
        System.out.println(x: "Logger instance created.");  
    }  
  
    // Public static method to provide access to the instance  
    public static Logger getInstance() {  
        if (instance == null) {  
            instance = new Logger();  
        }  
        return instance;  
    }  
  
    // Sample logging method  
    public void log(String message) {  
        System.out.println("LOG: " + message);  
    }  
}
```

Exercise 2: Implementing the Factory Method Pattern

```
c:\Users\Lohitha\OneDrive\Desktop\Digital_Nurture_Cts\Week_1\Design_and_principal>java SingletonTest  
Logger instance created.  
LOG: Application started.  
LOG: Processing data.  
logger1 hashCode: 1933863327  
logger2 hashCode: 1933863327  
Only one Logger instance exists.
```

```
package FactoryMethodPatternExample;

public interface Document {
    void open();
}
```

```
package FactoryMethodPatternExample;

public class PdfDocument implements Document {

    @Override
    public void open() {
        System.out.println(x: "PDF Document Opened");
    }
}
```

```
package FactoryMethodPatternExample;

public class ExcelDocument implements Document {

    @Override
    public void open() {
        System.out.println(x: "Excel Document Opened");
    }
}
```

```
Week_1 > Design_and_principal > FactoryMethodPatternExample > WordDocument
1 package FactoryMethodPatternExample;
2
3 public class WordDocument implements Document {
4
5     @Override
6     public void open() {
7         System.out.println(x: "Word Document Opened");
8     }
9 }
```

```
Week_1 > Design_and_principal > FactoryMethodPatternExample > DocumentFactory
package FactoryMethodPatternExample;

public abstract class DocumentFactory {

    public abstract Document createDocument();
}
```

```
ek_1 > Design_and_principal > FactoryMethodPatternExample > ExcelFactory.java > ExcelFact
1 package FactoryMethodPatternExample;
2
3 public class ExcelFactory extends DocumentFactory {
4
5     @Override
6     public Document createDocument() {
7         return new ExcelDocument();
8     }
9 }
10
```

```
ek_1 > Design_and_principal > FactoryMethodPatternExample > PdfFactory.java > PdfFactory
1 package FactoryMethodPatternExample;
2
3 public class PdfFactory extends DocumentFactory {
4
5     @Override
6     public Document createDocument() {
7         return new PdfDocument();
8     }
9 }
10
```

```
1 package FactoryMethodPatternExample;
2
3 public class WordFactory extends DocumentFactory {
4
5     @Override
6     public Document createDocument() {
7         return new WordDocument();
8     }
9 }
10
```

```

1  package FactoryMethodPatternExample;
2
3  public class FactoryTest {
4
5      Run | Debug
6      public static void main(String[] args) {
7
8          DocumentFactory wordFactory = new WordFactory();
9          Document wordDoc = wordFactory.createDocument();
10         wordDoc.open();
11
12         DocumentFactory pdfFactory = new PdfFactory();
13         Document pdfDoc = pdfFactory.createDocument();
14         pdfDoc.open();
15
16         DocumentFactory excelFactory = new ExcelFactory();
17         Document excelDoc = excelFactory.createDocument();
18         excelDoc.open();
19     }
}

```

```

c:\Users\Lohitha\OneDrive\Desktop\Digital_Nurture_Cts\Week_1\Design_and_principal>java FactoryMethodPatternExample.FactoryTest
Word Document Opened
PDF Document Opened
Excel Document Opened

```